# C. F. G. S. Desarrollo de Aplicaciones Web - Distancia -
# Examen de Programación de la 1º Evaluación. 25-11-2025

***"First, solve the problem. Then, write the code."*** **- John Johnson –**

1) Un número n es triangular cuando es posible expresarlo como la suma de varios números naturales consecutivos comenzando en 1.

   **Ejemplos:**
   - **10** → 10 = 1 + 2 + 3 + 4 → *es triangular*
   - **8** → no puede formarse como suma consecutiva → *no es triangular*
   - **15** → 15 = 1 + 2 + 3 + 4 + 5 → *es triangular*

   Pedir un número por teclado, obligando al usuario a que introduzca un valor impar comprendido entre 10 y 1000, ambos incluidos. En el caso de que el valor leído no cumpla estos requisitos, se visualizará el mensaje de error "número incorrecto, tiene que ser impar y comprendido entre 10 y 1000", y se volverá a pedir otro valor hasta que sea válido.

   **Ejemplo de ejecución del programa (utilizar el mismo formato):**

```
Introduce un numero impar entre 10 y 1000: 46
Numero incorrecto, tiene que ser impar y comprendido entre 10 y 1000.
Introduce un numero impar entre 10 y 1000: 91
El numero 91 es triangular.
```

2) Escribe un programa que lea tres números enteros por teclado: un límite inferior, un límite superior y un divisor.
   El programa debe mostrar, en orden descendente, todos los números comprendidos entre los dos límites (incluidos) que sean múltiplos del divisor y además sean números pares.
   Los valores mostrados deben ir separados por punto y coma y el último debe terminar con un punto.
   Si no existe ningún número que cumpla las condiciones, debe mostrarse el mensaje:
   "No hay valores que cumplan las condiciones."

   **Ejemplo de ejecución del programa (utilizar el mismo formato):**

```
Introduce el primer limite:
2
Introduce el segundo limite:
23
Introduce el divisor:
4

20; 16; 12; 8; 4.
```

3) Escribir un programa que pida por teclado un número entero impar n, y que a continuación dibuje una figura como la siguiente (ejemplo para n = 7).

```
*******
 *     *
  *   *
    *
```

Hay que obligar al usuario a que introduzca un valor impar, **utilizando un bucle for** para ello.

**Ejemplo de ejecución del programa (utilizar el mismo formato):**

```
Introduce un numero entero impar: 12
El numero debe ser impar. Intentalo de nuevo.
Introduce un numero entero impar: 5
*****
 * *
  *
```

4) Hacer un programa que pida por teclado una cadena de caracteres, y que asigne a otra cadena el resultado de replicar todas las apariciones del primer carácter de la cadena tantas veces como indique el último carácter de la misma. En este proceso no se tienen en cuenta el primer y último carácter. No es necesario comprobar que la cadena leída tiene el formato correcto.

El programa tiene que ejecutarse hasta que se teclee fin, tanto en mayúsculas como minúsculas.

**Ejemplo de ejecución del programa (utilizar el mismo formato):**

Introduce una cadena (o 'fin' para salir):
aabracadabra4
Resultado: aaaaabraaaacaaaadaaaabraaaa
Introduce una cadena (o 'fin' para salir):
bballena3
Resultado: bbbballena
Introduce una cadena (o 'fin' para salir):
fin
Programa finalizado.

**Puntuación de cada ejercicio:**

| | |
|---|---|
| Ejercicio 1: | 2 puntos |
| Ejercicio 2: | 2,5 puntos |
| Ejercicio 3: | 3 puntos |
| Ejercicio 4: | 2,5 puntos |

**Criterios de corrección y cómo entregar el examen:**

Aunque un programa no funcione, entregarlo igual, ya que puede ocurrir que esté bien planteado y falle por un pequeño detalle. De la misma manera, si se pide hacer un programa con una determinada sentencia para implementar un bucle y no os sale, hacerlo de otra. Evidentemente no puntuará lo mismo, pero al menos podéis completar el ejercicio.

Hay que subir la solución del examen como tarea, en formato de procesador de texto en un único fichero, que incluya el código fuente completo de cada uno de los ejercicios. La tarea está antes de la UD1 con el nombre “Examen de a 1ª evaluación”. Nombrar al documento con vuestro nombre y apellidos.

## AYUDA PARA EL EXAMEN

A la hora de programar, netbeans nos ayuda a escribir código de manera más rápida mediante decenas de atajos (shortcuts). NetBeans incluye un archivo pdf con la lista completa de dichos atajos. Esta lista se encuentra dentro del directorio de instalación de NetBeans en una carpeta llamada “nb”. Allí encontraremos 2 pdfs:

shortcuts.pdf

shortcuts_mac.pdf

El primero contiene los atajos para PC (Windows/Linux), el segundo para los usuarios de Mac.

Acordaros dónde está el fichero shortcuts.pdf para poder consultarlo el día del examen. Os recomiendo que os familiaricéis con el uso de estos atajos. De todas maneras, os pongo a continuación algunos de los atajos más útiles:

1. sout + Tab, genera la línea: System.out.println("");
2. psvm genera un método main

```
public static void main(String[] args) {

  }
```

3. sw + Tab, genera una sentencia switch

```
switch (var) {
        case val:

          break;
        default:
          throw new AssertionError();
      }
```

4. for, while, if +ctrl+spacio dan sugerencias para construir cualquiera de estas estructuras de control

Tampoco es necesario saberse de memoria todos los métodos de cada una de las clases que tiene el lenguaje. Para eso podemos consultar la API del lenguaje y los IDEs nos ayudan. Si no recordamos como se calcula el valor absoluto de un valor, simplemente escribiendo Math. (clase utilizada en los apuntes), nos aparecen todos los métodos de la clase, con lo que podemos localizar el que nos interesa utilizar. De la misma manera, si después de una variable de tipo String pongo el punto también me aparecen todos los métodos (extraer un carácter, convertir a mayúsculas…..).

```
Math.
    E                                          double
    PI                                         double
    IEEEremainder(double d, double d1)         double
    abs(double d)                              double
    abs(float f)                               float
    abs(int i)                                 int
    abs(long l)                                long
    acos(double d)                             double
    asin(double d)                             double
    atan(double d)                             double
    atan2(double d, double d1)                 double
    cbrt(double d)                             double
    ceil(double d)                             double
```

```
String nombre;

nombre.
    charAt(int i)                              char
    codePointAt(int i)                         int
    codePointBefore(int i)                     int
    codePointCount(int i, int i1)              int
    compareTo(String string)                   int
    compareToIgnoreCase(String string)         int
    concat(String string)                      String
    contains(CharSequence cs)                  boolean
    contentEquals(CharSequence cs)             boolean
```